

```
--- djangoapi\core\myLib\geoModelSerializer.py ---
```

```
from django.db import connection
from django.contrib.gis.geos import GEOSGeometry
```

```
from rest_framework import serializers
```

```
from djangoapi.settings import EPSG_FOR_GEOMETRIES, ST_SNAP_PRECISION
from .geometryTools import WkbConversor, GeometryChecks
```

```
class GeoModelSerializer(serializers.ModelSerializer):
```

```
    """
```

```
    This class is a serializer for models that have a geometry field.
```

```
    The geometry field is expected to be in WKT format, or GEOJSON.
```

```
    The the serializer expects the the table to have the fields:
```

```
        -id: the primary key of the table
```

```
        -geom: the geometry field
```

```
    The serializer will return the geometry in WKT and GEOJSON format,
```

```
    in the fields 'geom_geojson' and 'geom_wkt'.
```

```
    """
```

```
    check_geometry_is_valid = True #if true it will check if the geometry is valid: not
self-intersecting and closed
```

```
    check_st_relation = True #if true it will check the relation of the geometry with the
other geometries
```

```
    matrix9IM = 'T*****' #matrix 9IM for the relation of the geometries: 'T*****'
= interiors intersects
```

```
    geom = serializers.CharField(write_only=True) # Este campo es para input. No se
devuelve en GETs
```

```
    geom_geojson = serializers.SerializerMethodField() # Este campo es para
serialización (output)
```

```
    geom_wkt = serializers.SerializerMethodField() # Este campo es para serialización
(output)
```

```
    class Meta:
```

```
        fields = ['id', 'geom', 'geom_geojson', 'geom_wkt']
```

```
    def validate_geom(self, value):
```

```
        """Validates if a geometry in geojson/wkt is valid.
```

```
        If pass all checks, return the wkb value, which is the
```

```
        value stored in the database
```

```
        """
```

```
        print('validate_geom')
```

```
        c=WkbConversor()
```

```
        wkb=c.set_wkt_from_text(value)
```

```
        gc=GeometryChecks(wkb)
```

```
        if self.check_geometry_is_valid:
```

```
            if not gc.is_geometry_valid():
```

```
                raise serializers.ValidationError('Invalid geometry. May be
self-intersecting or not closed.')
```

```
        if self.check_st_relation:
```

```
            #we have to know if we are editing (UPDATE) or inserting (CREATE)
```

```
            if self.instance:
```

```

        print("It is an UPDATE. You must remove the current geometry from the
checks")

        gc.check_st_relate(self.get_table_name(),self.matrix9IM,
self.instance.id)
        if gc.are_there_related_ids():
            raise serializers.ValidationError(gc.get_relate_message())
        else:
            print("It is a CREATE.")
            gc.check_st_relate(self.get_table_name(),self.matrix9IM)
            if gc.are_there_related_ids():
                raise serializers.ValidationError(gc.get_relate_message())
        return wkb

def get_geom_geojson(self, obj):
    """Obtiene la geometría en formato WKT a partir de WKB usando PostGIS."""
    print('get_geom_asgeojson ')
    return obj.geom.geojson

def get_geom_wkt(self, obj):
    """Obtiene la geometría en formato WKT a partir de WKB usando PostGIS."""
    print('get_geom_wkt')
    return obj.geom.wkt

def get_table_name(self):
    return self.Meta.model._meta.db_table

class GeomodelPolygonSerializer(GeoModelSerializer):
    class Meta:
        fields = GeoModelSerializer.Meta.fields + ['area', 'perimeter']

    def validate(self, data):
        """
        Este método se ejecuta después de las validaciones de los campos validate_...().
        Cada validate devuelve un valor que puede haber sido modificado, estos valores
están en
        el objeto 'data' de este método.
        Aquí se calculan el área y el perímetro de la geometría
        """
        geom_modificada = data.get('geom')
        g=GEOSGeometry(geom_modificada, srid=EPSG_FOR_GEOMETRIES)
        data['area']=g.area

        data['perimeter']=g.length
        return data

class GeomodelLinestringSerializer(GeoModelSerializer):
    """
    Se espera que la tabla tenga el campo length
    :var fields: Descripción
    """
    class Meta:
        fields = GeoModelSerializer.Meta.fields + ['length']

```

```

def validate(self, data):
    """
    Este método se ejecuta después de las validaciones de los campos validate_...().
    Cada validate devuelve un valor que puede haber sido modificado, estos valores
    están en
    el objeto 'data' de este método.
    Aquí se calculan el área y el perímetro de la geometría
    """
    geom_modificada = data.get('geom')
    g=GEOSGeometry(geom_modificada, srid=EPSG_FOR_GEOMETRIES)
    data['length']=g.length
    return data


class GeoModelSerializer2(serializers.ModelSerializer):
    """
    This class is a serializer for models that have a geometry field.
    The geometry field is expected to be in WKT format, ot GEOJSON.
    The the serializer expects the the table to have the fields:
    -id: the primary key of the table
    -geom: the geometry field
    The serializer will return the geometry in WKT and GEOJSON format,
    in the fields 'geom_geojson' and 'geom_wkt'.
    """
    check_geometry_is_valid = True #if true it will check if the geometry is valid: not
    self-intersecting and closed
    check_st_relation = True #if true it will chck the relation of the geometry with the
    other geometries
    matrix9IM = 'T*****' #matrix 9IM for the relation of the geometries: 'T*****'
    = interiors intersects
    geoms_as_wkt = True #if true the serializer expects the geometries in WKT format. If
    false, in geojson format

    geom = serializers.CharField(write_only=True) # Este campo es para input. No se
    devuelve en GETs
    geom_geojson = serializers.SerializerMethodField() # Este campo es para
    serialización (output)
    geom_wkt = serializers.SerializerMethodField() # Este campo es para serialización
    (output)

    class Meta:
        fields = ['id', 'geom', 'geom_geojson', 'geom_wkt']

    def create(self, validated_data):
        geom_in_text = validated_data.pop('geom') # Obtener la geometría en formato
        geojson/wkt
        validated_data['geom'] = self.convert_to_wkb(geom_in_text)
        return super().create(validated_data)

    def update(self, instance, validated_data):
        if 'geom' in validated_data:
            geom_in_text = validated_data.pop('geom')

```

```

        validated_data['geom'] = self.convert_to_wkb(geom_in_text)
    return super().update(instance, validated_data)

def validate_geom(self, value):
    """Validates if a geometry in geojson is valid."""
    print('validate_geom')
    geom_binary = self.convert_to_wkb(value)

    if self.check_geometry_is_valid:
        if not self.is_geometry_valid(geom_binary):
            raise serializers.ValidationError('Invalid geometry. May be
self-intersecting or not closed.')

    """Validates if a geometry of the layer has the relation 'T*****' (interiors
intersects)."""
    if self.check_st_relation:
        r = self.check_st_relate(geom_binary)
        if len(r) > 0:
            table_name = self.get_table_name()
            raise serializers.ValidationError(f'Invalid geometry. Exist geometries
in the layer {table_name} with the relation {self.matrix9IM}. Ids: {r}.')
    return value

def get_geom_geojson(self, obj):
    """Obtiene la geometría en formato WKT a partir de WKB usando PostGIS."""
    print('get_geom_asgeojson 4')
    return self.get_geometry_as_geojson(obj.id)

def get_geom_wkt(self, obj):
    """Obtiene la geometría en formato WKT a partir de WKB usando PostGIS."""
    print('get_geom_wkt')
    return self.get_geometry_as_wkt(obj.id)

def get_geometry_as_geojson(self, model_id):
    """Returns the geometry as geojson from PostGIS."""
    table_name = self.get_table_name()
    print('get_geometry_as_geojson', table_name)
    with connection.cursor() as cursor:
        cursor.execute(f"SELECT ST_AsGeojson(geom) FROM {table_name} WHERE id = %s",
[model_id])
        row = cursor.fetchone()
        return row[0] if row else None # Devuelve la geometría en formato geojson o
None

def get_geometry_as_wkt(self, model_id):
    """Returns the geometry as geojson from PostGIS."""
    table_name = self.get_table_name()
    print('get_geometry_as_wkt', table_name)
    with connection.cursor() as cursor:
        cursor.execute(f"SELECT ST_AsText(geom) FROM {table_name} WHERE id = %s",
[model_id])
        row = cursor.fetchone()
        return row[0] if row else None # Devuelve la geometría en formato geojson o

```

None

```
def convert_geojson_to_wkb(self, geojson_value):
    with connection.cursor() as cursor:
        #Ejecuta la función PostGIS ST_GeomFromText para convertir WKT a WKB
        print('convert_geojson_to_wkb')
        q="""SELECT
            ST_SNAPTOGRID(
                st_setsrid(
                    ST_GeomFromGeoJSON(%s)
                    ,%s
                ),
                %s
            )
        """
        cursor.execute(q, [geojson_value, EPSG_FOR_GEOMETRIES, ST_SNAP_PRECISION])
        row = cursor.fetchone()
        return row[0] # Esto será el WKB

def convert_wkt_to_wkb(self, wkt_value):
    with connection.cursor() as cursor:
        #Ejecuta la función PostGIS ST_GeomFromText para convertir WKT a WKB
        print('convert_wkt_to_wkb')
        q="""SELECT
            ST_SNAPTOGRID(
                st_setsrid(
                    ST_GeomFromText(%s)
                    ,%s
                ),
                %s
            )
        """
        cursor.execute(q, [wkt_value, EPSG_FOR_GEOMETRIES, ST_SNAP_PRECISION])
        row = cursor.fetchone()
        return row[0] # Esto será el WKB

def is_geometry_valid(self, geom_binary):
    """Checks if a geometry in geojson is valid."""
    print('is_geometry_valid')
    with connection.cursor() as cursor:
        q="""SELECT ST_IsValid(%s)"""
        cursor.execute(q, [geom_binary])
        row = cursor.fetchone()
        #row is true or false
        return row[0]

def get_table_name(self):
    return self.Meta.model._meta.db_table

def check_st_relate(self, geom_binary: str, layerName: str= None, matrix9IM: str =
None):
    """Checks whether or not exists a geometry with the relation of the geom with all
the geometries in the layer
```

```

        layername using the matrix 9IM. The geom is in geojson format.
    """
    if layerName is None:
        layerName = self.get_table_name()

    if matrix9IM is None:
        matrix9IM = self.matrix9IM

    print('check_st_relate')

    with connection.cursor() as cursor:
        q=f"""SELECT id FROM {layerName} WHERE ST_relate(geom,%s,%s)"""
        cursor.execute(q, [geom_binary, matrix9IM])
        row = cursor.fetchall()
        return row # Devuelve los ids de las geometrías que cumplen la relación

def convert_to_wkb(self, geom_in_text: str):
    """Converts a geometry in text format to binary format."""
    if self.geoms_as_wkt:
        return self.convert_wkt_to_wkb(geom_in_text)
    else:
        return self.convert_geojson_to_wkb(geom_in_text)

```